



# Chapter 13

## Project 4.1: Visual Analysis Techniques

When doing **exploratory data analysis (EDA)**, one common practice is to use graphical techniques to help understand the nature of data distribution. The US **National Institute of Standards and Technology (NIST)** has an *Engineering Statistics Handbook* that strongly emphasizes the need for graphic techniques. See <https://doi-org.ezproxy.sfpl.org/10.18434/M32189>.

This chapter will create some additional Jupyter notebooks to present a few techniques for displaying univariate and multivariate distributions.

In this chapter, we'll focus on some important skills for creating diagrams for the cleaned data:

- Additional Jupyter Notebook techniques
- Using **PyPlot** to present data
- Unit testing for Jupyter Notebook functions

This chapter has one project, to build the start of a more complete analysis notebook. A notebook can be saved and exported as a PDF file, allowing an analyst to share preliminary results for early conversations. In the next chapter, we'll expand on the notebook to create a presentation that can be shared with colleagues.

Looking further down the road, a notebook can help to identify important aspects of the data that need ongoing monitoring. The computations created here will often become the basis for more fully automated reporting tools and notifications. This analysis activity is an important step toward understanding the data and designing a model for the data.

We'll start with a description of an analysis notebook.

## 13.1 Description

In the previous chapters, the sequence of projects created a pipeline to acquire and then clean the raw data. The intent is to build automated data gathering as Python applications.

We noted that ad hoc data inspection is best done with a notebook, not an automated CLI tool. Similarly, creating command-line applications for analysis and presentation can be challenging. Analytical work seems to be essentially exploratory, making it helpful to have immediate feedback from looking at results.

Additionally, analytical work transforms raw data into information, and possibly even insight. Analytical results need to be shared to create significant value. A Jupyter notebook is an exploratory environment that can create readable, helpful presentations.

One of the first things to do with raw data is to create diagrams to illustrate the distribution of univariate data and the relationships among variables in multivariate data. We'll emphasize the following common kinds of diagrams:

- **Histograms** A histogram summarizes the distribution of values for a variable in a dataset. The histogram will have data values on one axis and frequency on the other axis.
- **Scatter Plots** A scatter plot summarizes the relationships between values for two variables in a dataset. The visual clustering can be

apparent to the casual observer.

For small datasets, each relationship in a scatter plot can be a single dot. For larger datasets, where a number of points have similar relationships, it can be helpful to create “bins” that reflect how many points have the same relationship.

There are a number of ways of showing the size of these bins. This includes using a color code for more popular combinations. For some datasets, the size of an enclosing circle can show the relative concentration of similarly-valued data. The reader is encouraged to look at alternatives to help emphasize the interesting relationships among the attributes of the various samples.

The use of **Seaborn** to provide colorful styles is also important when working with diagrams. You are encouraged to explore various color palettes to help emphasize interesting data.

## 13.2 Overall approach

We’ll take some guidance from the C4 model ( <https://c4model.com>) when looking at our approach:

- **Context:** For this project, the context diagram has two use cases: the acquire-to-clean process and this analysis notebook.
- **Containers:** There’s one container for analysis application: the user’s personal computer.
- **Components:** The software components include the existing analysis models that provide handy definitions for the Python objects.
- **Code:** The code is scattered in two places: supporting modules as well as the notebook itself.

A context diagram for this application is shown in ***Figure 13.1***.

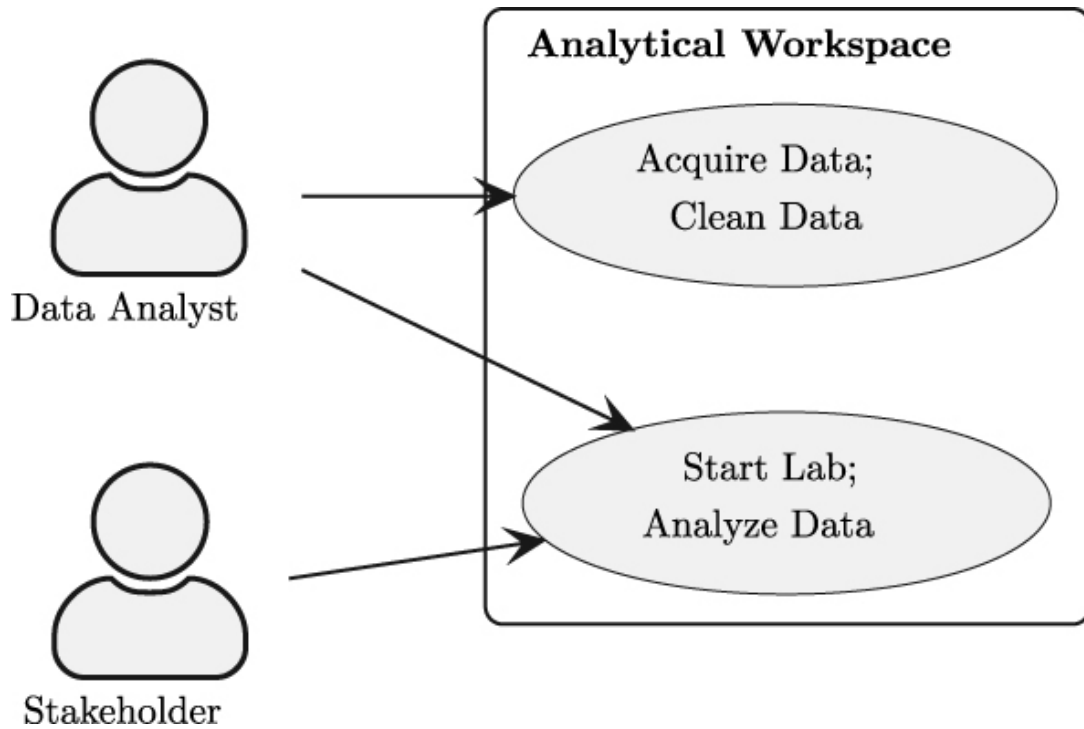


Figure 13.1: Context diagram

The analyst will often need to share their analytical results with stakeholders. An initial notebook might provide confirmation that some data does not conform to the null hypothesis, suggesting an interesting relationship that deserves deeper exploration. This could be part of justifying a budget allocation to do more analysis based on preliminary results. Another possible scenario is sharing a notebook to confirm the null hypothesis is likely true, and that the variations in the data have a high probability of being some kind of measurement noise. This could be used to end one investigation and focus on alternatives.

In ***Chapter 6, Project 2.1: Data Inspection Notebook***, the data inspection notebook was described. The use of a single acquisition model module for the data was mentioned, but the details of the Python implementation weren't emphasized. The raw data module often provides little useful structure to the data when doing inspections.

Moving forward into more complicated projects, we'll see the relationship between the notebooks and the modules that define the data model become more important.

For this analysis notebook, the analysis data model, created in [\*\*\*Chapter 9, Project 3.1: Data Cleaning Base Application\*\*\*](#), will be a central part of the notebook for doing analysis. This will be imported and used in the analysis notebook. The analysis process may lead to changes to the analysis model to reflect lessons learned.

A technical complication arises from the directory structure. The data acquisition and cleaning applications are in the `src` directory, where the notebooks are kept in a separate `notebooks` directory.

When working with a notebook in the `notebooks` directory, it is difficult to make the Python `import` statement look into the adjacent `src` directory. The `import` statement scans a list of directories defined by the `sys.path` value. This value is seeded from some defined rules, the current working directory, and the value of the `PYTHONPATH` environment variable.

There are two ways to make the `import` statement load modules from the adjacent `src` directory:

1. Put `../src` into the `PYTHONPATH` environment variable before starting JupyterLab.
2. Put the absolute path to `../src` into the `sys.path` list after starting JupyterLab.

The two are equivalent. The first can be done by updating one's `~/.zshrc` file to make sure the `PYTHONPATH` environment variable is set each time a terminal session starts. There are other files appropriate for other shells; for example, `~/.bashrc` or `./rc` for the classic `sh` shell. For Windows, there's a dialog that allows one to edit the system's environment variables.

The alternative is to update `sys.path` with a cell containing code like the following example:

```
import sys
from pathlib import Path

src_path = Path.cwd().parent / "src"
sys.path.append(str(src_path))
```

This cell will add the peer `../src` directory to the system path. After this is done, the `import` statement will bring in modules from the `../src` directory as well as bringing in built-in standard library modules, and modules installed with **conda** or **pip**.

An initialization module can be defined as part of the IPython startup. This module can alter the `sys.path` value in a consistent way for a number of related notebooks in a project.

While some developers object to tinkering with `sys.path` in a notebook, it has the advantage of being explicit.

Setting `PYTHONPATH` in one's `~/.zshrc` file is a very clean and reliable solution. It then becomes necessary to put a reminder in a `README` file so that new team members can also make this change to their personal home directory.

When sharing notebooks, it becomes imperative to make sure all stakeholders have access to the entire project the notebook depends on. This can lead to a need to create a Git repository that contains the notebook being shared along with reminders, test cases, and needed modules.

Once we have the path defined properly, the notebook can share classes and functions with the rest of the applications. We'll move on to looking at one possible organization of an analysis notebook.

### 13.2.1 General notebook organization

An analytical notebook is often the primary method for presenting and sharing results. It differs from a “lab” notebook. It's common for lab notebooks to contain a number of experiments, some of which are fail-

ures, and some of which have more useful results. Unlike a lab notebook, an analytical notebook needs to have a tidy organization with carefully-written Markdown cells interspersed with data processing cells. In effect, an analysis notebook needs to tell a kind of story. It needs to expose actors and actions and the consequences of those actions.

It's essential that the notebook's cells execute correctly from beginning to end. The author should be able to restart the kernel and run all of the cells at any time to redo the computations. While this may be undesirable for a particularly long-running computation, it still must be true.

A notebook may have preliminary operations that aren't relevant to most readers of the report. A specific example is setting the `sys.path` to import modules from an adjacent `../src` directory. It can be helpful to make use of JupyterLab's ability to collapse a cell as a way to set some of the computation details aside to help the reader focus on the key concepts.

It seems sensible to formalize this with Markdown cells to explain the preliminaries. The remaining cells in this section can be collapsed visually to minimize distractions.

The preliminaries can include technical and somewhat less technical aspects. For example, setting `sys.path` is purely technical, and few stakeholders will need to see this. On the other hand, reading the `SeriesSample` objects from an ND JSON format file is a preliminary step that's somewhat more relevant to the stakeholder's problems.

After the preliminaries, the bulk of the notebook can focus on two topics:

- Summary statistics
- Visualizations

We'll look at the summary statistics in the next section.

### 13.2.2 Python modules for summarizing

For initial reports, Python's `statistics` module offers a few handy statistical functions. This module offers `mean()`, `median()`, `mode()`, `stdev()`, `variance()`. There are numerous other functions here the reader is encouraged to explore.

These functions can be evaluated in a cell, and the results will be displayed below the cell. In many cases, this is all that's required.

In a few cases, though, results need to be truncated to remove meaningless trailing digits. In other cases, it can help to use an f-string to provide a label for a result. A cell might look like the following:

```
f"mean = {statistics.mean(data_values): .2f}"
```

This provides a label, and truncates the output to two places to the right of the decimal point.

In some cases, we might want to incorporate computed values into markdown cells. The **Python Markdown** extension provides a tidy way to incorporate computed values into markdown content.

See <https://jupyter-contrib-nbextensions.readthedocs.io/en/latest/nbextensions/python-markdown/readme.html>.

The most important part of the notebook is the graphic visualization of the data, we'll turn to that in the next section.

### 13.2.3 PyPlot graphics

The **matplotlib** package works well for creating images and diagrams. Within this extensive, sophisticated graphic library is a smaller library,



`pyplot`, that's narrowly focused on data visualizations. Using the `pyplot` library permits a few lines of code to create a useful display of data.

The module is often renamed to make it easier to type. The common convention is shown in the following line of code:

```
import matplotlib.pyplot as plt
```

This lets us use `plt` as a namespace to refer to functions defined in the `pyplot` module of the `matplotlib` package.

In some cases, JupyterLab may not have the `matplotlib` library prepared for interactive use. (This will be clear when the plotted images are not shown in the notebook.) In these cases, the interactive use of the `matplotlib` library needs to be enabled. Use the following magic commands in a cell of the notebook to enable interactive use:

```
%matplotlib inline  
%config InlineBackend.figure_formats = {'png', 'retina'}
```

The first command enables the `matplotlib` library to create graphics immediately in the notebook. It won't create separate files or pop-up windows. The second command produces readily-shared PNG output files. It also helps **MAC OS X** users to optimize the graphics for their high-resolution displays.

This isn't always required. It's needed only for those installations that didn't configure the `matplotlib` library for interactive use in a Jupyter notebook.

In many cases, the `pyplot` library expects simple sequences of values for the various plotting functions. The *x* and *y* values of a scatter plot, for example, are expected to be two parallel lists of the same length. This will lead to a few extra cells to restructure data.

The data cleaning applications from previous chapters produced a single sequence of compound sample objects. Each object was a separate sample, with the related values for all of the variables. We'll need to convert from this sample record organization to parallel sequences of values for the individual variables.

This can involve something like the following:

```
x = [s.x for s in series_1]
y = [s.y for s in series_1]
```

An alternative is to use the `operator.attrgetter()` function. It looks like the following:

```
from operator import attrgetter

x = list(map(attrgetter('x'), series_1))
y = list(map(attrgetter('y'), series_1))
```

We'll start with a histogram to show the distribution of values for a single variable.

## Data frequency histograms

Often, a document — like a book — has individual figures. A figure may contain a single plot. Or, it may contain a number of “subplots” within a single figure. The `matplotlib` library provides support for creating a single figure that contains many subplots. The idea of a figure with a single plot can be seen as a special case of this generalized approach.

A figure with a single plot can be prepared with a statement like the following:

```
fig, ax = plt.subplots()
```

The `fig` object is the figure as a whole. The `ax` object is the set of axes for the subplot within the figure. While the explicit `ax` object can seem unnecessary, it's part of a more general approach that allows figures with related plots to be built.

The more general case stacks multiple subplots within a figure. The `subplots()` function can return axes for each subplot. The call to create a two-plot figure might look like this:

```
fig, (ax_0, ax_1) = plt.subplots(1, 2)
```

Here we've stated we'll have 1 row with 2 subplots stacked next to each other. The `fig` object is the figure as a whole. The `ax_0` and `ax_1` objects are the sets of axes for each of these two subplots within the figure.

Here's an example of creating a single histogram from the `y` values of one of the four Anscombe's quartet series:

```
fig, ax = plt.subplots()

# Labels and Title
ax.set_xlabel('Y')
ax.set_ylabel('Counts')
ax.set_title('Series I')

# Draw Histogram
_ = ax.hist(y, fill=False)
```

For this example, the `y` variable must be a list of `y` attribute values from Series I. The series data must be read from the cleaned source file.

The `_ = ax.hist(...)` statement assigns the results of the `hist()` function to the variable `_` as a way to suppress displaying the result value in this cell. Without this assignment statement, the notebook will show the result of the `hist()` function, which isn't very interesting

and clutters up the output. Since each series has both an  $x$  and a  $y$  value, it helps to stack two histograms that shows the distribution of these values. The reader is encouraged to develop a figure with two stacked subplots.

The number of options for color and borders on the histogram bars are breathtaking in their complexity. It helps to try several variants in a single notebook and then delete the ones that aren't helpful.

The comparison between values is often shown in a scatter plot that shows each  $(x,y)$  pair. We'll look at this next.

## X-Y scatter plot

A scatter plot is one of many ways to show the relationship between two variables. Here's an example of creating a single plot from the  $x$  and  $y$  values of one of the four Anscombe's quartet series:

```
fig, ax = plt.subplots()

# Labels and Title
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_title('Series I')

# Draw Scatter
_ = ax.scatter(x, y)
```

The `_ = ax.scatter(...)` statement assigns the results of the `scatter()` function to the variable `_` as a way to suppress displaying the value. This keeps the output focused on the figure.

The above example presumes the data have been extracted from a sequence of `SeriesSample` instances using code like the following:

```
x = [s.x for s in series_1]
```

```
y = [s.y for s in series_1]
```

This, in turn, presumes the value for `series_1` was read from the clean data created by the **acquire** and **clean** applications.

Now that we have an approach to building the notebook, we can address the way notebooks tend to evolve.

### 13.2.4 Iteration and evolution

A notebook is often built iteratively. Cells are added, removed, and modified as the data is understood.

On a purely technical level, the Python programming in the cells needs to evolve from a good idea to software that's testable. It's often easiest to do this by rewriting selected cells into functions. For example, define a function to acquire the ND JSON data. This can be supplemented by `doctest` comments to confirm the function works as expected.

A collection of functions can be refactored into a separate, testable module if needed. This can permit wider reuse of good ideas in multiple notebooks.

Equally important is to avoid “over-engineering” a notebook. It's rarely worth the time and effort to carefully specify the contents of a notebook, and then write code that meets those specifications. It's far easier to create — and refine — the notebook.

In some large organizations, a senior analyst may direct the efforts of junior analysts. In this kind of enterprise, it can be helpful to provide guidance to junior analysts. When formal methods are needed, the design guidance can take the form of a notebook with markdown cells to explain the desired goal.

Now that we have an approach to building the notebook, we can enumerate the deliverables for this project.

## 13.3 Deliverables

This project has the following deliverables:

- A `requirements-dev.txt` file that identifies the tools used, usually `jupyterlab==3.5.3` and `matplotlib==3.7.0`.
- Documentation in the `docs` folder.
- Unit tests for any new application modules in the `tests` folder.
- Any new application modules in the `src` folder with code to be used by the inspection notebook.
- A notebook to summarize the clean data. In the case of Anscombe's quartet, it's essential to show the means and variances are nearly identical, but the scatter plots are dramatically different.

We'll look at a few of these deliverables in a little more detail.

### 13.3.1 Unit test

There are two distinct kinds of modules that can require testing:

- The notebook with any function or class definitions. All of these definitions require unit tests.
- If functions are factored from the notebook into a supporting module, this module will need unit tests. Many previous projects have emphasized these tests.

A notebook cell with a computation cell is notoriously difficult to test. The visual output from the `hist()` or `scatter()` functions seems almost impossible to test in a meaningful way.

In addition, there are numerous usability tests that can't be automated. Poor choice of colors, for example, can obscure an important relationship. Consider the following questions:

1. Is it informative?
2. Is it relevant?

### 3. Is it misleading in any way?

In many cases, these questions are difficult to quantify and difficult to test. As a consequence, it's best to focus automated testing on the Python programming.

It's imperative to avoid testing the internals of `matplotlib.pyplot`.

What's left to test?

- The data loading.
- Any ad hoc transformations that are part of the notebook.

The data loading should be reduced to a single function that creates a sequence of `SeriesSample` instances from the lines in an ND JSON file of clean data. This loading function can include a test case.

We might define the function as follows:

```
def load(source_file):
    """
    >>> from io import StringIO
    >>> file = StringIO('{"x": 2.0, "y": 3.0}\\n{"x": 5.0, "y": 7.0}')
    >>> d = load(file)
    >>> len(d)
    2
    >>> d[0]
    SeriesSample(x=2.0, y=3.0)
    """
    data = [
        SeriesSample(**json.loads(line)) for line in source_file if line
    ]
    return data
```

This permits testing by adding a cell to the notebook that includes the following:

```
import doctest
doctest.testmod()
```

This cell will find the functions defined by the notebook, extract any doctest cases from the docstrings in the function definitions, and confirm the doctest cases pass.

For more complicated numerical processing, the **hypothesis** library is helpful. See [\*Hypothesis testing\*](#) in [\*Chapter 10, Data Cleaning Features\*](#) for more information.

### 13.3.2 Acceptance test

An automated acceptance test is difficult to define for a notebook. It's hard to specify ways in which a notebook is helpful, meaningful, or insightful in the simple language of Gherkin scenarios.

The `jupyter execute <filename>` command will execute an `.ipynb` notebook file. This execution is entirely automated, allowing a kind of sanity check to be sure the notebook runs to completion. If there is a problem, the command will exit with a return code of 1, and the cell with the error will be displayed in detail. This can be handy for confirming the notebook isn't trivially broken.

The `.ipynb` file is a JSON document. An application (or a step definition for **Behave**) can read the file to confirm some of its properties. An acceptance test case might look for error messages, for example, to see if the notebook failed to work properly.

Cells with `"type": "code"` will also have `"outputs"`. If one of the outputs has `"output_type": "error"`; this cell indicates a problem in the notebook. The notebook did not run to completion, and the acceptance test should be counted as a failure.

We can use projects like **Papermill** to automate notebook refresh with new data. This project can execute a template notebook and save the



results as a finalized output notebook with values available and computations performed.

For more information, see <https://papermill.readthedocs.io>.

## 13.4 Summary

This project begins the deeper analysis work on clean data. It emphasizes several key skills, including:

- More advanced Jupyter Notebook techniques. This includes setting the `PYTHONPATH` to import modules and creating figures with plots to visualize data.
- Using **PyPlot** to present data. The project uses popular types of visualizations: histograms and scatter plots.
- Unit testing for Jupyter Notebook functions.

In the next chapter, we'll formalize the notebook into a presentation “slide deck” that can be shown to a group of stakeholders.

## 13.5 Extras

Here are some ideas for the reader to add to these projects.

### 13.5.1 Use Seaborn for plotting

An alternative to the **pyplot** package is the **Seaborn** package. This package also provides statistical plotting functions. It provides a wider variety of styling options, permitting more colorful (and perhaps more informative) plots.

See <https://seaborn.pydata.org> for more information.

This module is based on `matplotlib`, making it compatible with JupyterLab.

Note that the **Seaborn** package can work directly with a list-of-dictionary structure. This matches the ND JSON format used for acquiring and cleaning the data.

Using a list-of-dictionary type suggests it might be better to avoid the analysis model structure, and stick with dictionaries created by the **clean** application. Doing this might sacrifice some model-specific processing and validation functionality.

On the other hand, the `pydantic` package offers a built-in `dict()` method that covers a sophisticated analysis model object into a single dictionary, amenable to use with the **Seaborn** package. This seems to be an excellent way to combine these packages. We encourage the reader to explore this technology stack.

### 13.5.2 Adjust color palettes to emphasize key points about the data

Both the **matplotlib** package and the **Seaborn** package have extensive capabilities for applying color to the plot. A choice of colors can sometimes help make a distinction visible, or it can obscure important details.

You can consider alternative styles to see which seems more useful.

In some enterprise contexts, there are enterprise communications standards, with colors and fonts that are widely used. An important technique when using Seaborn is to create a style that matches enterprise communication standards.

A number of websites provide website color schemes and design help. Sites like <https://paletton.com> or [colordesigner.io](https://colordesigner.io) provide complementary color palettes. With some effort, we can take these kinds of designs for color palettes and create Seaborn styles that permit a consistent and unique presentation style.

